

A Major Project Report on
AI BASED STOCK MARKET PREDICTION USING A STACKED
LSTM

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by
IV B.Tech II Semester

P. SHAHIRA	223H1A3910
A. NANDINI	223H1A3901
K. VARA LAKSHMI	223H1A3905
M. ZAIBA KHANAM	223H1A3907
S. AFIFA FATIMA	223H1A3911

Under the Esteemed Guidance of

MS. S.SWETHA, (M. Tech)

Assistant Professor, Dept. of AI&ML



DEPARTMENT
OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
KANDULA LAKSHUMMA MEMORIAL COLLEGE OF ENGINEERING FOR
WOMEN

(Approved by AICTE New Delhi, Recognized under section 2(f) of the UGC Act, 1956. Affiliated to
JNTUA, Anantapuramu, Andhra Pradesh)

KADAPA-5160023

2025-2026

KANDULA LAKSHUMMA MEMORIAL COLLEGE OF ENGINEERING FOR WOMEN

(Approved by AICTE New Delhi, Recognized under section 2(f) of the UGC Act, 1956. Affiliated to JNTUA, Anantapuramu, Andhra Pradesh)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

This is to certify that the project report entitled **“AI BASED STOCK MARKET PREDICTION USING A STACKED LSTM ”** that is being submitted by

P. SHAHIRA	223H1A3910
A. NANDINI	223H1A3901
K. VARALAKSHMI	223H1A3905
M. ZAIBA KHANAM	223H1A3907
S. AFIFA FATIMA	223H1A3911

In partial fulfilment of the requirement for the award of the Degree of the Bachelor of Technology In ARTIFICIAL INTELLIGENCE AND DATA SCIENCE to the **Jawaharlal Nehru Technological University Anantapur** is a record of bona fide work carried out under my guidance and supervision. The results embodied in this project report have not been submitted in any university or Organization for the award of any degree.

Internal Guide

MS. S. SWETHA, (M. Tech)
Assistant Professor

Head of the Department

MR. B. MAHESH NAYAK, M. Tech,(Ph. D)
Assistant Professor& HOD

Submitted for university Examination (Viva Voice) held on_____

Internal Examiner

External Examiner

VISION AND MISSION OF COLLEGE

VISION

To be an Institution which aims at empowering women through delivering quality Technical Education while keeping Moral Integrity intact.

MISSION

- To provide high standard of Technical Education to women students enabling them to face all challenges of their personal and professional life with courage and confidence.
- To make women students as global citizens by inculcating critical thinking, effective communication, robust engineering knowledge and ethical behaviour in them.
- To provide an environment that has values and encourages knowledge acquisition, gender equality and academic freedom, making this a preferred Institution for knowledge seekers.
- To install self confidence among rural students by supplementing with co-curricular and extra-curricular activities.

VISION AND MISSION OF DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VISION

To generate competent professionals to become part of the industry and research organizations at the National and International levels.

MISSION

- Providing a strong theoretical and practical background across the computer science discipline with an emphasis on software development.
- Imparting the skills necessary to continue education and grow professionally. Empowering the women in rural communities with computer education.
- Inculcating professional behavior, strong ethical values, innovative research capabilities and leadership abilities.

DECLARATION

I hereby declare that this project report titled "AI Based Stock Market Prediction - DL Based Stacked LSTM" submitted by me is based on my own work carried out during the project period in the Department of Artificial Intelligence and Machine Learning. I have not submitted this work for any degree or diploma to any university or educational institution. The project work represents my original contribution in the field of Artificial Intelligence and Machine Learning, specifically Deep Learning, and it is applied in the Financial Technology domain for Stock Market Prediction.

Date: _____ Place: _____

Signature of Student(s): _____

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our Project Supervisor S. Swetha, M.Tech, Assistant Professor, for providing us an opportunity to undertake this project work and for their valuable guidance, encouragement and support throughout the course of this project. Their expert guidance and constructive suggestions have helped us in the successful completion of this project. We also wish to thank our Head of Department B. Mahesh Nayak, M.Tech (Ph.D.) and faculty members who have directly or indirectly guided and helped in various aspects of this project work.

We are grateful to the Department of Artificial Intelligence and Machine Learning and K.L.M College of Engineering for Women for providing us with the necessary resources and facilities to carry out this project. We also thank our friends and family for their support and encouragement. We acknowledge the contributions of various researchers and developers whose work has inspired and informed this project.

Project Associates:

P. Shahira (223H1A3910)

A. Nandini (223H1A3901)

K. Vara Lakshmi (223H1A3905)

S. Afifa Fatima (223H1A3911)

M. Zaiba Khanam (223H1A3907)

ABSTRACT

Stock market prediction is a complex and challenging task due to the dynamic and volatile nature of financial markets. Investors often find it difficult to make accurate decisions because stock prices are influenced by multiple factors. With the advancement of Artificial Intelligence and Machine Learning, predicting stock trends has become more efficient.

This project focuses on developing an AI-based system for stock market prediction and analysis. The system uses historical stock data to identify patterns and trends in price movements. A deep learning model called Long Short-Term Memory (LSTM) is used for accurate time-series forecasting. LSTM is capable of learning long-term dependencies in sequential data. The model is trained using past stock prices and tested to predict future values. The application collects real-time stock data using financial APIs.

It also calculates technical indicators such as Moving Averages and Relative Strength Index (RSI). These indicators help in understanding market conditions and generating trading signals. An interactive dashboard is developed using Streamlit for user-friendly visualization. Users can view stock trends, charts, and prediction results in real time. The system also provides buy, sell, or hold signals based on analysis. Model performance is evaluated using metrics like MAE and RMSE. A 7-day forecast is generated to help users plan their investments. This project demonstrates the application of AI in the financial domain and helps users make informed and data-driven investment decisions.

Keywords: Stock Market Prediction, LSTM, Deep Learning, Stacked LSTM, Streamlit, Time-Series Forecasting, Technical Indicators, MAE, RMSE.

TABLE OF CONTENTS

Chapter Contents

#

	Certificate	1-2.
	Vision and Mission	3
	Declaration	4.
	Acknowledgement.....	5
	Abstract	6.
	List of Figures.....	10
1	Introduction	11.
	1.1 Overview of Stock Market Prediction.....	11
	1.2 Problem Statement.....	12.
	Existing System and Disadvantages.....	13
	2.1 Existing Market System.....	13
	2.1.1 Disadvantages of Existing System	13
	Proposed System and Advantages	14
	3.1 Stock Market Prediction Proposed Solution	15
	3.1.1 Advantages of Proposed System	15
	System Design and Architecture.....	16.
	4.1 System Architecture Overview	17
	4.1.1 System Architecture Diagram.....	18
2	Software Methodologies.....	19
	2.1 Introduction.....	20
	2.2 waterfall model flowchart.....	21
	2.2 Waterfall Model Phases.....	22

3	System Analysis.....	24.
	3.1 Feasibility Study.....	25
	3.2 Functionality Requirements.....	26..
4	System Specifications	27
	4.1 Hardware Requirements	28
	4.2 Software Requirements.....	28
5	System Design (UML Diagrams).....	
	295.1 Introduction.....	30
	5.1.1 Use Case Diagram	31.
	5.1.2 Collaboration Diagram.....	33
	5.1.3 Activity Diagram	34
	5.1.4 Sequence Diagram.....	36
	5.1.5 Deployment Diagram.....	38
6	Implementations and Results.....	
	396.1 Introduction – Python	40
	6.2 Algorithm – Stacked LSTM.....	41
	6.3 Source of code.....	42-53
	6.4 Screenshots	54
7	Testing.....	62
	71 System Testing	63
	7.2 Testing Techniques.....	64
8	Conclusion and Future Enhancement.....	65
	8.1 Conclusion	66
	.8.2 Future Enhancements.....	67
	.	

9	References	68-69
10	Annexure	70
	10.1 annexure a.....	71
	10.2 annexure b.....	71
	10.3 annexure c.....	71
	10.4 annexure d.....	71
	10.5 annexure e.....	72
	10.6 annexure f.....	72

LIST OF FIGURES

Fig No.	Title	Page No.
4.1	System Architecture Diagram	18
8.1	Use Case Diagram	31
8.2	Collaboration Diagram	33
8.3	Activity Diagram	34
8.4	Sequence Diagram	36
8.5	Deployment Diagram	38
9.1	Sample Dataset – Stock Price History (Yahoo Finance)	55
9.2	User Interface – Login System	56
9.3	Trading Dashboard with Market Overview	57
9.4	Candlestick Chart with Moving Averages (MA20, MA50)	58
9.5	Price Distribution Chart	59
9.6	AI Prediction Dashboard – Actual vs Predicted Prices	60
9.7	7-Day Forecast with Download Option	61

Chapter -1
INTRODUCTION

1.INTRODUCTION

1.1 Overview of Stock Market Prediction

Stock market prediction is a complex and challenging task due to the dynamic and volatile nature of financial markets. Investors often find it difficult to make accurate decisions because stock prices are influenced by multiple factors such as economic conditions, political events, company performance, and investor sentiment. With the advancement of Artificial Intelligence and Machine Learning, predicting stock trends has become more efficient and accessible.

This project focuses on developing an AI-based system for stock market prediction and analysis. The system uses historical stock data to identify patterns and trends in price movements. A deep learning model called Long Short-Term Memory (LSTM) is used for accurate time-series forecasting. LSTM is capable of learning long-term dependencies in sequential data, making it particularly well-suited for financial time-series analysis. The model is trained using past stock prices and tested to predict future values. The application collects real-time stock data using financial APIs such as yFinance.

An interactive dashboard is developed using Streamlit for user-friendly visualization. Users can view stock trends, charts, and prediction results in real time. The system also provides buy, sell, or hold signals based on technical analysis. Model performance is evaluated using metrics like Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). A 7-day forecast is generated to help users plan their investments. This project demonstrates the practical application of AI in the financial domain and helps users make informed and data-driven investment decisions.

1.2 Problem Statement

The stock market is highly volatile and unpredictable, making it difficult for investors to make accurate decisions. Traditional methods of stock analysis are time-consuming and depend on manual interpretation, which may lead to errors. Additionally, the large volume of financial data generated daily makes it challenging to analyze trends effectively and identify the right time to buy or sell stocks.

With the advancement of Artificial Intelligence and Deep Learning, there is a need for an intelligent system that can analyze historical data and predict future stock prices. This project aims to develop a real-time stock prediction system using Long Short-Term Memory (LSTM) networks to provide

accurate forecasts and assist users in making informed investment decisions. The system addresses the gap in existing tools by combining deep learning prediction with real-time data fetching and interactive visualization in a single unified platform.



2. EXISTING SYSTEM AND DISADVANTAGES

2.1 Existing Market System

The existing stock market systems mainly rely on traditional methods such as fundamental and technical analysis, where investors manually study charts, historical data, and financial reports to make decisions. These systems are often time-consuming and require expert knowledge to interpret market trends accurately. Most available platforms provide basic visualization tools but lack advanced predictive capabilities. Additionally, they do not efficiently handle large volumes of real-time data or utilize modern technologies like Artificial Intelligence and Machine Learning. As a result, investors frequently depend on limited insights or assumptions, which can lead to inaccurate predictions and poor investment decisions.

2.1.1 Disadvantages of Existing System

- **Time Consumption:** The system requires manual analysis of stock data and charts, which is time-consuming and inefficient.
- **Requires Expert Knowledge:** Users need strong financial knowledge to understand and interpret market trends accurately.
- **Human Errors:** Manual analysis can lead to mistakes and incorrect decision-making, resulting in financial losses.
- **Lack of Prediction Accuracy:** Existing systems do not provide reliable or accurate future stock predictions based on data patterns.
- **Limited Use of Modern Technology:** They do not effectively use advanced technologies like Artificial Intelligence and Machine Learning for analysis.
- **Poor Handling of Big Data:** These systems struggle to process large volumes of real-time stock market data efficiently.
- **No Automation:** Decision-making is not automated, making the process slow and inefficient for time-sensitive markets.
- **Limited Insights:** They provide only basic information, which may lead to poor investment decisions without comprehensive analysis.

3.PROPOSED SYSTEM AND ADVANTAGES

3.1 Stock Market Prediction – Proposed Solution

The proposed system is an AI-based stock market prediction and analysis platform that utilizes advanced technologies such as Artificial Intelligence and Machine Learning to provide accurate and efficient predictions. It employs a deep learning model, specifically Long Short-Term Memory (LSTM), to analyze historical stock data and identify patterns for forecasting future prices. The system collects real-time stock data through financial APIs such as yFinance and processes it to generate meaningful insights.

It also incorporates technical indicators like Moving Averages (MA20, MA50) and Relative Strength Index (RSI) to understand market trends and provide trading signals. A user-friendly interactive dashboard is developed using Streamlit to visualize stock performance through candlestick charts, pie charts, and prediction graphs. Additionally, the system automates decision-making by providing buy, sell, or hold suggestions based on the analysis. Model performance is evaluated using metrics such as MAE and RMSE, and a 7-day forecast is generated to assist users in making informed and data-driven investment decisions.

3.1.1 Advantages of Proposed System

- **High Prediction Accuracy:** Provides more accurate stock predictions using Artificial Intelligence and the Stacked LSTM deep learning model.
- **Efficient Data Analysis:** Uses Machine Learning to analyze large volumes of historical and real-time data quickly and efficiently.
- **Advanced Forecasting Model:** Implements Stacked Long Short-Term Memory (LSTM) for better time-series prediction by capturing complex sequential patterns.
- **Automation:** Reduces manual effort by automating data analysis, preprocessing, model training, and prediction processes.
- **Real-Time Data Processing:** Fetches and processes live stock data via yFinance API for up-to-date insights and analysis.
- **Decision Support System:** Provides buy, sell, and hold signals based on technical indicators to assist users in decision-making.

- **User-Friendly Interface:** Offers an interactive and easy-to-use dashboard developed using Streamlit for seamless user interaction.
- **Data Visualization:** Displays stock trends through candlestick charts, moving averages, and pie charts for better understanding.
- **Performance Evaluation:** Evaluates model performance using standard metrics like MAE and RMSE for transparency and reliability.
- **Future Forecasting:** Generates a 7-day stock price forecast with a downloadable CSV output for better investment planning.
- **Time Saving:** Reduces the time required for manual analysis by automating the entire prediction pipeline.
- **Reduced Human Errors:** Minimizes mistakes by relying on automated and data-driven predictions rather than manual interpretation.

4. SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture Overview

The proposed system is implemented using a deep learning approach with a Stacked Long Short-Term Memory (LSTM) model to perform stock price prediction. Initially, historical stock data is collected using financial APIs (yFinance) and preprocessed using normalization techniques such as MinMaxScaler to bring the values into a suitable range of 0 to 1. The data is then converted into sequences of 60 time steps to capture temporal dependencies in stock price movements.

The core of the system is the Stacked LSTM architecture, where multiple LSTM layers are arranged sequentially to improve learning capability. The first LSTM layer is configured with 50 units and set to return sequences, allowing the next LSTM layer to receive sequential data. The second LSTM layer also contains 50 units and processes the extracted features further. Finally, a Dense layer with a single neuron is added to produce the predicted stock price output.

The model is compiled using the Adam optimizer and Mean Squared Error (MSE) as the loss function. It is trained on the prepared dataset with a defined number of epochs and batch size to optimize performance. After training, the model is used to predict stock prices on test data, and the results are transformed back to original values using inverse scaling.

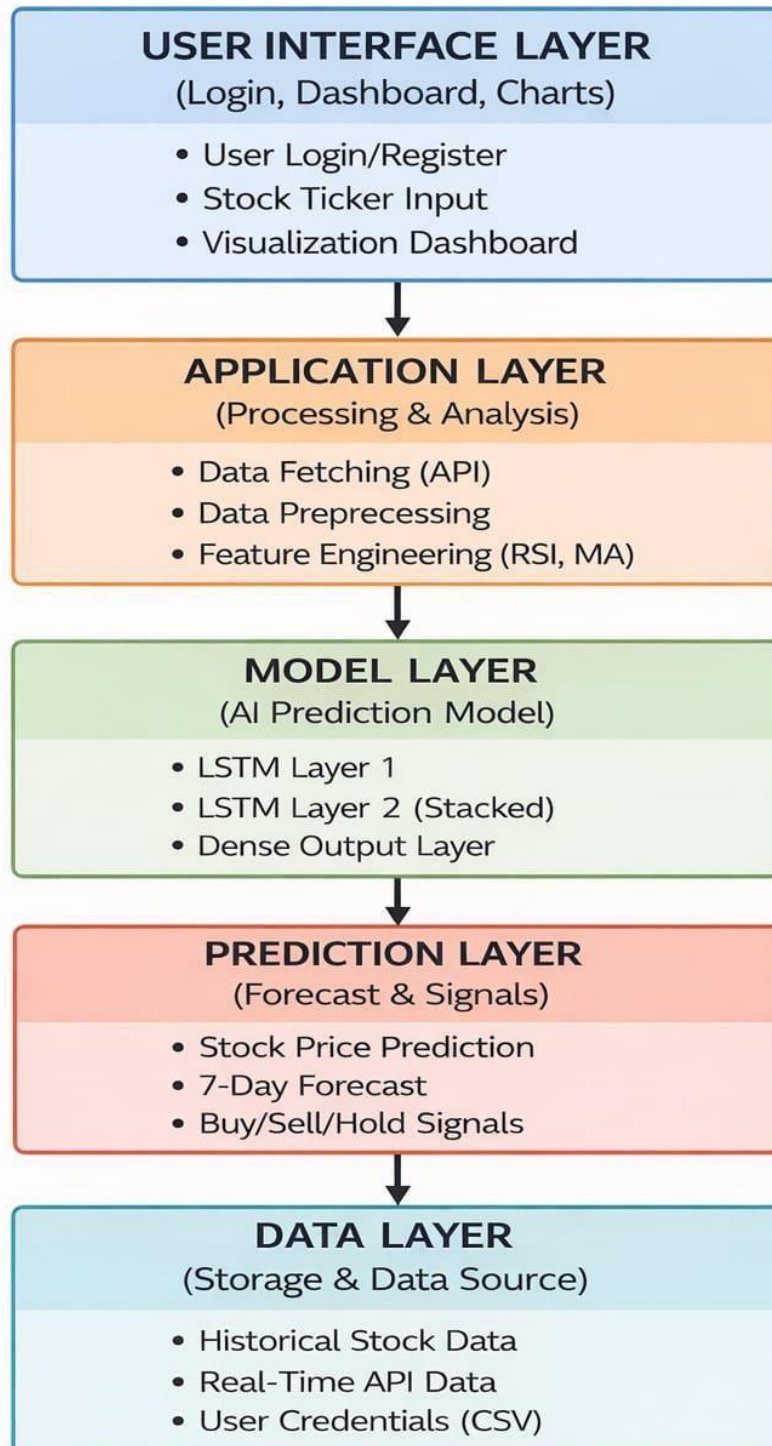
The system also performs future forecasting by iteratively predicting the next values based on previous outputs. This Stacked LSTM implementation enhances prediction accuracy by capturing complex patterns in time-series data. The integration of Artificial Intelligence and Deep Learning ensures efficient and reliable stock market prediction. Technical indicators such as Moving Averages and RSI are computed alongside the LSTM predictions to provide comprehensive trading signals.

4.1.1 System Architecture Diagram

The system follows a layered architecture as shown below:

SYSTEM ARCHITECTURE

(STOCK PREDICTION SYSTEM)



Chapter -2
SOFTWARE
METHODOLOGIES

2. SOFTWARE METHODOLOGIES

2.1 Introduction

The development of this project follows the Waterfall Model, which is a linear and sequential approach to software development. In this model, each phase is completed step by step, ensuring a structured and well-managed development process. The project begins with requirement analysis to understand the need for a stock prediction system, followed by system design where the architecture, user interface, and model structure are planned. In the implementation phase, the system is developed using Python, Streamlit, and deep learning techniques such as Stacked LSTM. The testing phase evaluates the model using metrics like MAE and RMSE to ensure accuracy. After successful testing, the system is deployed for user interaction through a dashboard, and finally, the maintenance phase ensures updates and improvements. This approach provides clarity, simplicity, and effective project management throughout the development process.

2.2 Waterfall Model Flowchart

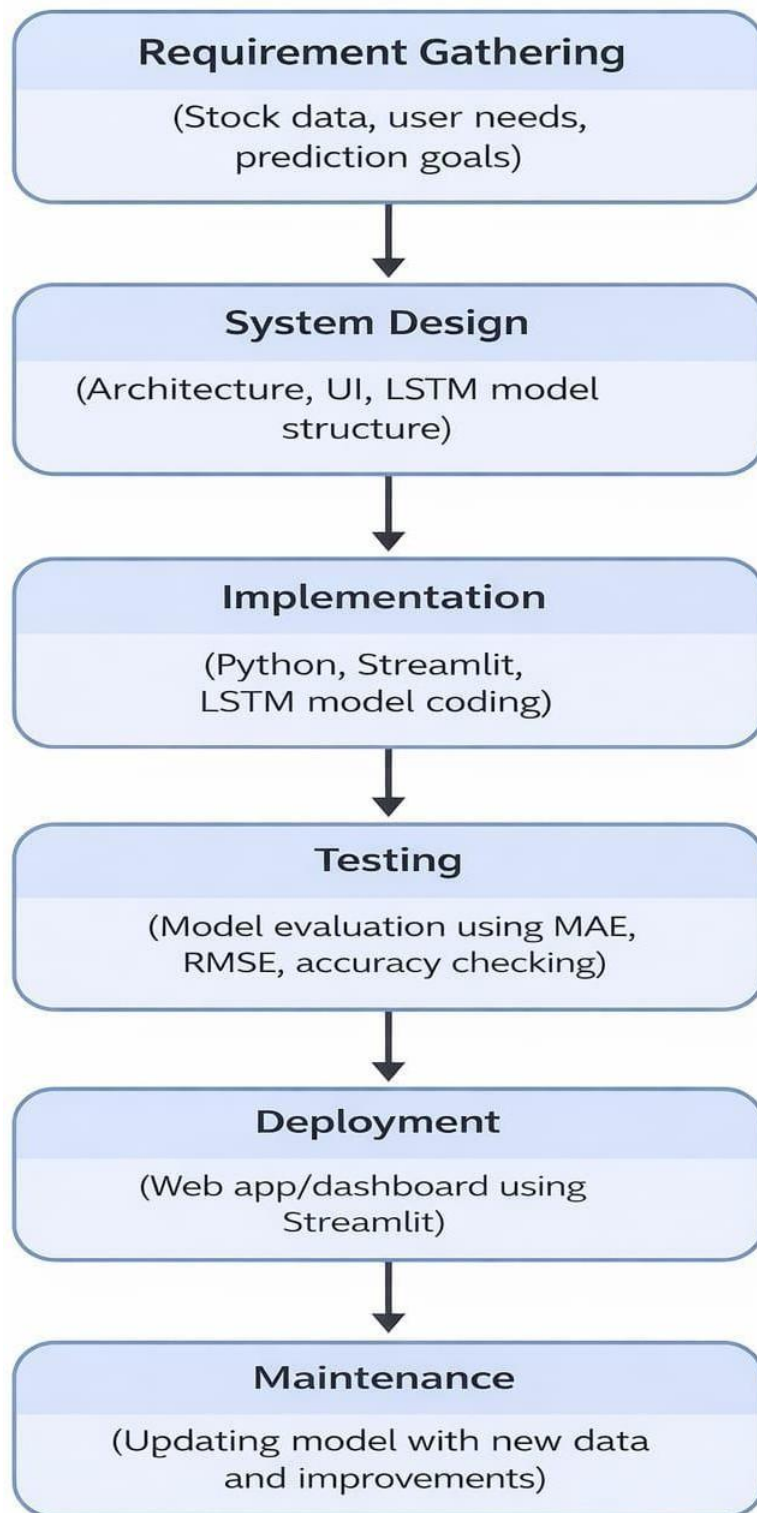


Fig: Waterfall Model

2.3 Waterfall Model Phases

The Waterfall Model consists of the following sequential phases:

- Requirement Gathering
- System Design
- Implementation
- Testing
- Deployment
- Maintenance

2.3.1 Requirement Gathering

In this stage, all necessary information such as stock data sources, user requirements, functional specifications, and prediction goals are collected and analyzed. Stakeholder needs are identified, including the need for real-time data fetching, prediction accuracy, and an interactive dashboard. The required tools, technologies, and datasets are also determined at this stage.

2.3.2 System Design

The overall system architecture is designed, including the user interface layout and the structure of the Stacked LSTM model used for prediction. The data flow from API to preprocessing to model training and output visualization is planned. Database design, API integration strategy, and the Streamlit dashboard layout are finalized during this phase. UML diagrams including use case, sequence, activity, deployment, and collaboration diagrams are created to document the design.

2.3.3 Implementation and Coding

The system is developed using Python, Streamlit, and deep learning techniques such as Stacked LSTM based on the designed architecture. The implementation includes: data collection using the yFinance API, data preprocessing and normalization using MinMaxScaler, LSTM model building and training using TensorFlow/Keras, technical indicator computation (MA20, MA50, RSI), Streamlit-based user interface development, and integration of all components into a unified application.

2.3.4 Testing

The system is tested to evaluate performance using metrics like MAE and RMSE, ensuring accuracy and reliability. Unit testing validates individual components such as data preprocessing functions and prediction methods. Integration testing ensures all modules work together correctly. Manual UI testing verifies usability and correct visualization of results. System testing validates the complete workflow from data input to prediction output.

2.3.5 Deployment

The completed system is deployed as a web application or dashboard for user interaction. The Streamlit application is launched locally or on a cloud platform, allowing users to enter stock ticker symbols and receive predictions in real time. The deployment ensures that the system is accessible, responsive, and performs efficiently under various input conditions.

2.3.6 Maintenance

The system is continuously updated with new data and improvements to enhance performance over time. Maintenance includes retraining the model with updated stock data, fixing any bugs identified during use, and incorporating new features such as additional technical indicators or support for more stock markets. Cloud deployment enables easier updates and continuous improvement of the system.

Chapter -3
SYSTEM
ANALYSIS

3. SYSTEM ANALYSIS

3.1 Feasibility Study

The feasibility study of the stock market prediction system evaluates the practicality of implementing the proposed solution using Python, Streamlit, and a deep learning model based on Long Short-Term Memory. Three key considerations are evaluated:

- Technical Feasibility
- Economic Feasibility
- Social Feasibility

3.1.1 Technical Feasibility

The project is technically feasible as it uses easily available tools like Python, Streamlit, TensorFlow, Keras, and Long Short-Term Memory. It does not require high-end hardware and can run on standard systems with a minimum of 4 GB RAM and an internet connection. All required technologies and data resources are accessible through open-source libraries and free financial APIs (yFinance), ensuring smooth implementation. The LBPH-like modular design of the system allows easy debugging and upgrades.

3.1.2 Economic Feasibility

The project is economically feasible as it uses open-source tools like Python, Streamlit, TensorFlow, Keras, and yFinance, reducing development costs to near zero. No expensive hardware or proprietary software is required for implementation. Hence, the overall cost of development and deployment is minimal. Cloud platforms like Heroku or Streamlit Cloud offer free tier deployment options, further reducing infrastructure costs.

3.1.3 Social Feasibility

The project is socially feasible as it is user-friendly and can be easily used by both beginners and experienced investors. It helps users make informed decisions through clear analysis, interactive charts, and AI-driven predictions. The system provides value to society by improving financial awareness and decision-making, potentially reducing the risk of poor investment choices. Its accessibility through a web browser makes it available to a wide demographic of users.

3.2 Functionality Requirements

3.1.2 Input Design

The input design of the system is simple and user-friendly. The user provides input through a web interface developed using Streamlit. The main input includes the stock ticker symbol (e.g., AAPL for Apple, TSLA for Tesla) entered through a text box. Additionally, users can select refresh intervals using a slider to update real-time data. Login and registration inputs such as username and password are also required for authentication. The input fields are validated to ensure correct and meaningful data is entered by the user. Invalid ticker symbols result in appropriate error messages guiding the user to correct input.

3.1.2 Output Design

The output design focuses on clear and interactive visualization of results. The system displays stock data through an interactive dashboard, including key metrics such as minimum, maximum, average, and closing prices. It presents candlestick charts, moving averages (MA20, MA50), and pie charts for better understanding of price distribution. The system also shows trading signals (buy/sell/hold) based on technical analysis. Predicted and actual stock prices are displayed using comparative line graphs, along with model evaluation metrics (MAE, RMSE, and Accuracy percentage). Additionally, a 7-day forecast is presented in tabular format, with an option to download the results as a CSV file for offline analysis.

Chapter -4
SYSTEM
SPECIFICATIONS

4.SYSTEM SPECIFICATIONS

4.1 Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB (8 GB recommended for better performance)
- Storage: At least 10 GB of free disk space
- Internet Connection: Required for fetching real-time stock data and APIs
- Webcam/Display: Standard monitor with minimum 1280x720 resolution

4.2 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Programming Language: Python 3.8 or above
- Libraries: Pandas, NumPy, Matplotlib, Plotly
- Web Framework: Streamlit
- ML Libraries: TensorFlow / Keras (for Stacked LSTM model)
- Data Source: yFinance API for real-time stock data
- Scaling: Scikit-learn (MinMaxScaler)
- IDE: VS Code / PyCharm / Jupyter Notebook
- Version Control: Git

Chapter -5
SYSTEM DESIGN

5. SYSTEM DESIGN

5.1 Introduction

UML (Unified Modelling Language) diagrams help visualize how the system is structured and how its components interact. The following UML diagrams are used to describe the AI Stock Market Prediction System:

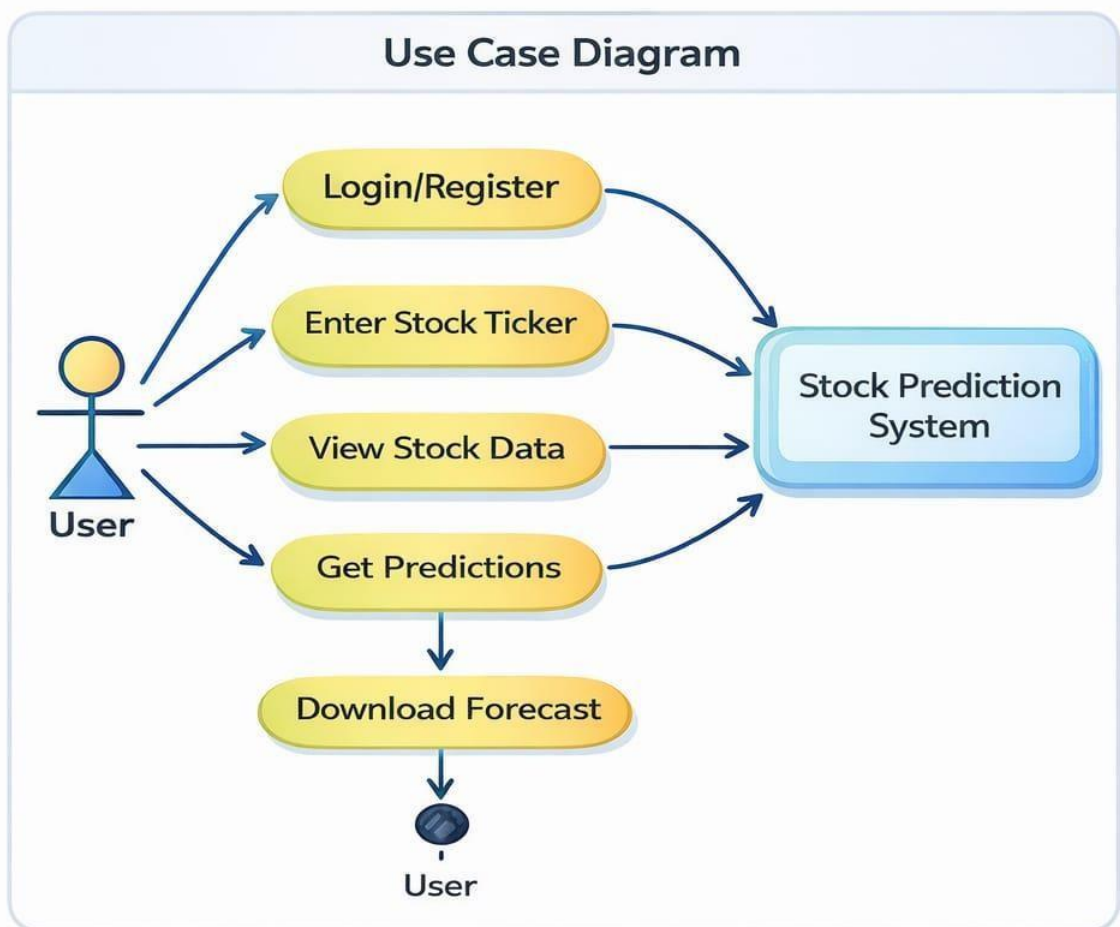
- Use Case Diagram: Represents the interaction between the user and the system functionalities. It shows how users perform actions like login, view data, and get predictions.
- Class Diagram: Represents the structure of the system by showing classes, attributes, and relationships. It helps in understanding how different components of the system are organized.
- Sequence Diagram: Shows the step-by-step interaction between system components over time. It describes how requests flow from the user to the system and back.
- Activity Diagram: Represents the workflow of the system processes. It shows the sequence of activities from input to output.
- Deployment Diagram: Shows the physical arrangement of hardware and software components. It represents how the system is deployed and executed.

These diagrams provide a clear system blueprint for development and maintenance.

5.1.1 Use Case Diagram

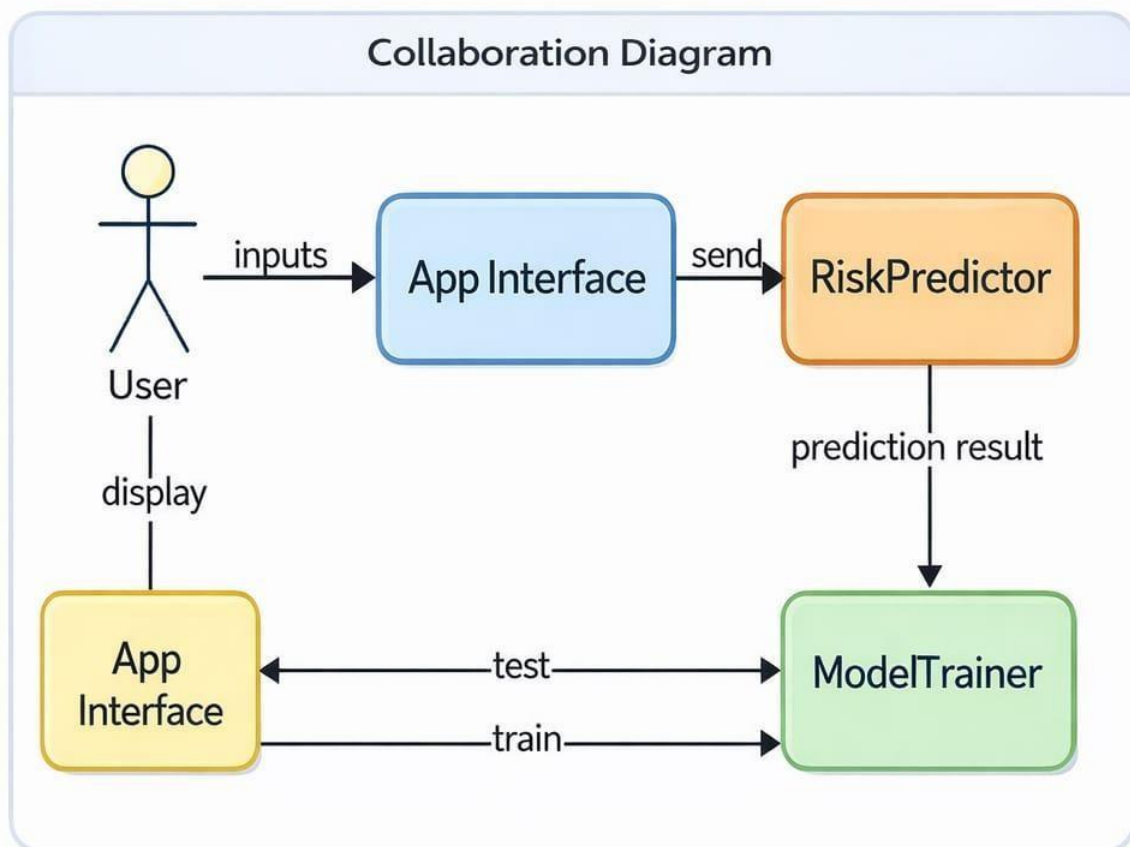
The Use Case Diagram shows the interactions between the User (Voter/Investor) and the Stock Prediction System. The primary actor is the User, and the system use cases are:

- Login / Register
- Enter Stock Ticker Symbol
- View Stock Data and Charts
- Get AI Predictions (Actual vs Predicted)
- View Trading Signals (Buy / Sell / Hold)
- Download 7-Day Forecast



5.1.2 Collaboration Diagram

The Collaboration Diagram illustrates how the User, App Interface, RiskPredictor, and ModelTrainer components interact. The user provides inputs to the App Interface, which sends data to the RiskPredictor and ModelTrainer for processing. The RiskPredictor computes technical indicators and trading signals, while the ModelTrainer handles LSTM training and prediction.



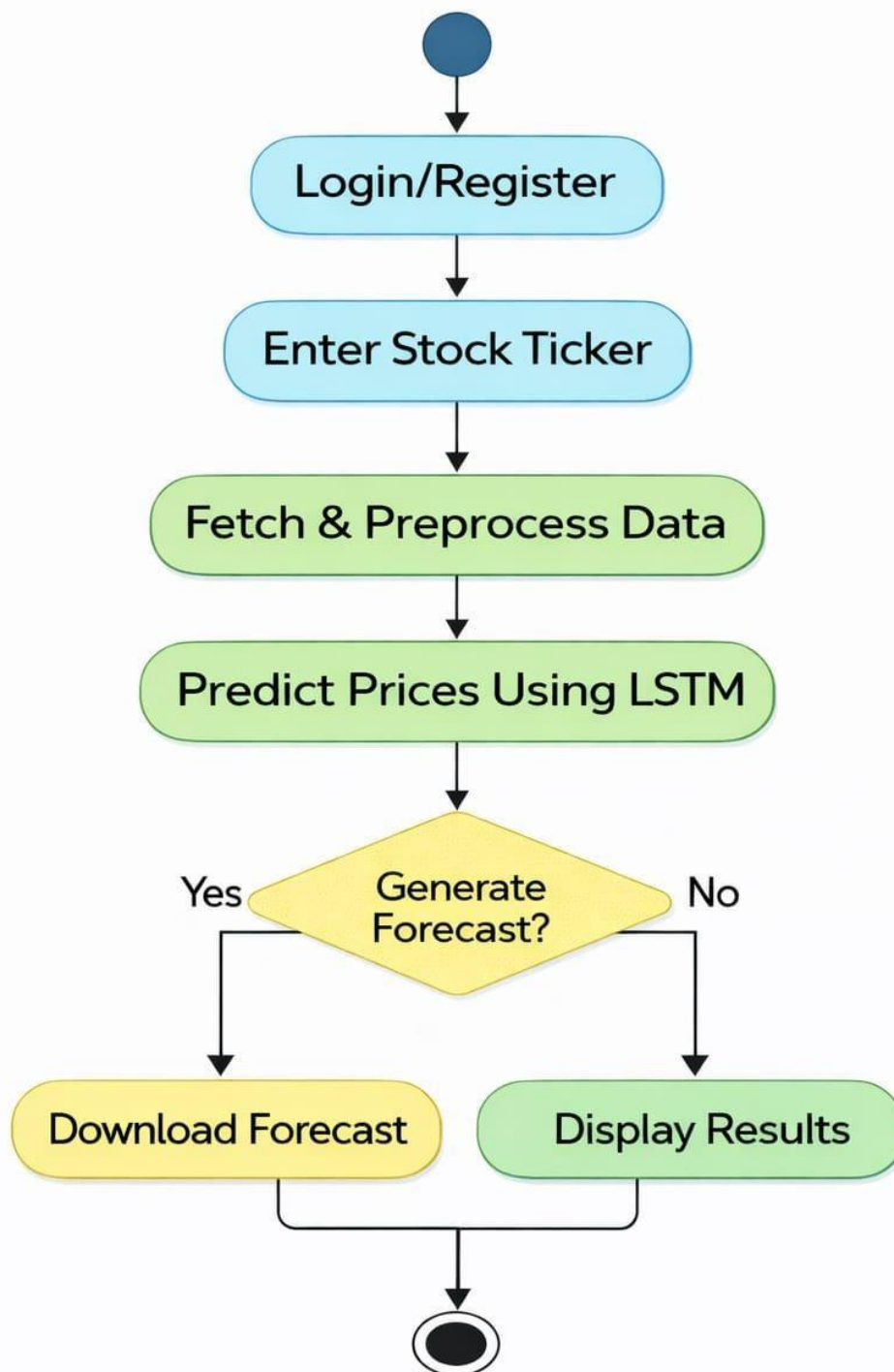


Fig 5.13 Activity Diagram

1. User opens the application and navigates to Login / Register page.
2. User enters credentials and authenticates.
3. User enters a Stock Ticker Symbol (e.g., AAPL, TSLA).
4. System fetches historical data from yFinance API.
5. Data is preprocessed (scaling, indicator computation, sequence creation).
6. Stacked LSTM model predicts stock prices.
7. System checks: Generate Forecast?
8. If yes – System generates 7-day forecast and offers CSV download.
9. Results, charts, metrics, and trading signals are displayed to the user.

5.1.4 Sequence Diagram

The Sequence Diagram for AI Stock Prediction using Machine Learning shows the step-by-step message flow between components:

1. User enters login credentials → Web Interface validates and authenticates.
2. User enters stock ticker → Web Interface sends ticker to ModelHandler.
3. ModelHandler fetches historical stock data from yFinance API.
4. Data is preprocessed: scaling with MinMaxScaler, computation of MA20, MA50, RSI.
5. Stacked LSTM model is trained on the prepared 60-step sequences.
6. Model predicts stock prices on test data; inverse scaling applied.
7. Prediction results, MAE, RMSE, and Accuracy metrics are computed.
8. Charts (candlestick, actual vs predicted, pie chart) and 7-day forecast are displayed to the user.

AI STOCK PREDICTION USING MACHINE LEARNING

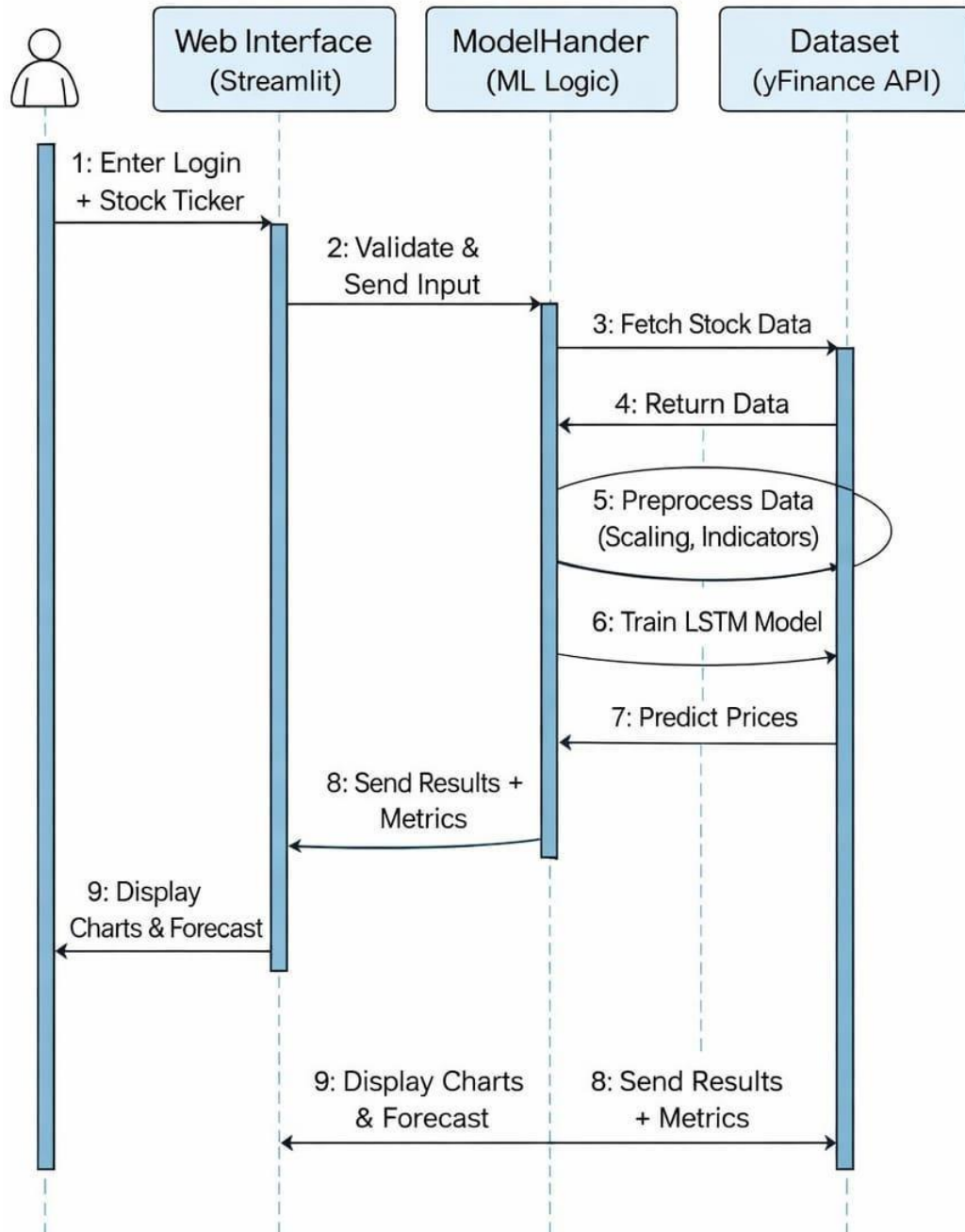


Fig 5.1.4 Sequence Diagram

5.1.5 Deployment Diagram

The Deployment Diagram shows the physical deployment of the AI Stock Prediction system:

- **User Device (Browser):** The user accesses the application through any modern web browser.
- **Streamlit Server (Web App):** The Streamlit application runs on a local or cloud server, handling the user interface and interactions.
- **Model Server (Stacked LSTM Model):** The TensorFlow/Keras LSTM model processes data and generates predictions on the server side.
- **yFinance API (External Data Source):** Real-time stock data is fetched from Yahoo Finance via the yFinance Python library.

AI STOCK PREDICTION DEPLOYMENT



Fig 5.1.5 Deployment Diagram

Chapter -6
IMPLEMENTATIO
NS AND RESULTS

6 IMPLEMENTATIONS AND RESULTS

6.1 Introduction – Python

Python is a high-level, interpreted, and general-purpose programming language created by Guido van Rossum in 1991. It is widely used for web development, data analysis, artificial intelligence, machine learning, and automation. Python is known for its simple syntax, which makes it easy to learn and use. In this project, Python is used to build the AI stock prediction system, handle data processing, and implement machine learning models like Stacked LSTM.

Key Features of Python used in this project:

- **Easy to Learn and Read:** Python uses simple English-like syntax, reducing the learning curve for new developers.
- **Interpreted Language:** Python executes code line-by-line, enabling quick testing and debugging during development.
- **Rich Libraries and Frameworks:** NumPy (numerical computations), Pandas (data handling and manipulation), TensorFlow/Keras (Machine Learning and Deep Learning), Matplotlib/Plotly (data visualization), Streamlit (web app UI), yFinance (real-time stock data fetching).
- **Supports Machine Learning and AI:** Python is the most popular language for AI/ML development, with extensive community support and documentation.
- **Platform Independent:** Python runs on Windows, Linux, and macOS without changing code, ensuring portability.
- **Rapid Development:** Due to fewer lines of code and high-level abstractions, projects can be developed faster compared to languages like Java or C++.
- **Integration Capability:** Python easily integrates with APIs, databases, cloud services, and other languages.
- **Object-Oriented and Functional:** Python supports multiple programming styles, allowing flexibility in design patterns.
- **Strong Community Support:** Millions of developers contribute to Python, ensuring a vast ecosystem of packages and libraries.

6.2 Algorithm – Stacked LSTM

The proposed system uses the Stacked Long Short-Term Memory (Stacked LSTM) algorithm for stock price prediction. Stacked LSTM is an advanced version of the traditional LSTM where multiple LSTM layers are stacked on top of each other to improve learning capability and prediction accuracy.

Stacked LSTM consists of multiple LSTM layers where the output of one LSTM layer becomes the input to the next layer. It captures complex patterns and long-term dependencies in time-series data and performs significantly better than a single-layer LSTM for stock market prediction tasks.

Why LSTM for Stock Prediction?

- Standard neural networks cannot retain information from previous time steps, making them unsuitable for sequential financial data.
- LSTM networks use memory cells and gating mechanisms (input gate, forget gate, output gate) to selectively remember or forget information over long sequences.
- This makes LSTM ideal for capturing patterns in stock price time series where past prices influence future prices.

In this project, the Stacked LSTM model uses:

- Input Layer: 60 time steps of normalized closing price data.
- First LSTM Layer: 50 units with `return_sequences=True` to pass sequential output to the next LSTM layer.
- Second LSTM Layer: 50 units to further extract complex temporal patterns.
- Dense Output Layer: 1 neuron to produce the final predicted stock price.
- Optimizer: Adam optimizer with Mean Squared Error (MSE) as the loss function.
- Normalization: MinMaxScaler used to scale data to [0, 1] range before training and inverse-scaled after prediction.

6.3 Source code

```
from evaluation import evaluate_model

import streamlit as st

import pandas as pd

import numpy as np

import yfinance as yf

import plotly.graph_objects as go

import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense

from streamlit_autorefresh import st_autorefresh

import os

from evaluation import evaluate_model # Make sure evaluation.py exists


# ----- PAGE CONFIG -----

st.set_page_config(page_title="AI Stock Prediction", layout="wide")


st.markdown("""

<style>

.block-container { max-width: 1100px; margin:auto; }
```

```
h1,h2,h3 { text-align:center; }
```

```
</style>
```

```
""", unsafe_allow_html=True)
```

```
#                USER DB
```

```
-----  
USER_FILE = "users.csv"
```

```
def load_users():
```

```
    if not os.path.exists(USER_FILE):
```

```
        pd.DataFrame(columns=["username","password"]).to_csv(USER_FILE,index=False)
```

```
    return pd.read_csv(USER_FILE)
```

```
def save_user(u,p):
```

```
    df = load_users()
```

```
    df =
```

```
    pd.concat([df,pd.DataFrame([[u,p]],columns=["username","password"])],ignore_index=True  
)
```

```
    df.to_csv(USER_FILE,index=False)
```

```
# ----- SESSION -----
```

```
if "login" not in st.session_state:
```

```
    st.session_state.login=False
```

```
if "logout_msg" not in st.session_state:
```

```
    st.session_state.logout_msg=""
```

```
if "refresh" not in st.session_state:
```

```
    st.session_state.refresh=15
```

```
if st.session_state.logout_msg:
```

```
    st.success(st.session_state.logout_msg)
```

```
    st.session_state.logout_msg = "👏 Logout successful! Thanks for exploring real-time AI  
stock insights. Come back anytime!"
```

```
# ----- LOGIN PAGE -----
```

```
if not st.session_state.login:
```

```
    if st.session_state.logout_msg:
```

```
        st.success(st.session_state.logout_msg)
```

```
        st.session_state.logout_msg=""
```

```
col1,col2,col3 = st.columns([2,3,2])
```

```
with col1:
```

```
    st.image("https://img.freepik.com/free-vector/stock-market-concept-  
illustration_114360-1686.jpg")
```

with col2:

```
st.markdown("<h1>🤖 AI STOCK PREDICTION</h1>",unsafe_allow_html=True)
```

```
menu = st.radio("Select Option",["Login","Register"])
```

```
users_df = load_users()
```

```
if menu=="Login":
```

```
    u = st.text_input("Username")
```

```
    p = st.text_input("Password",type="password")
```

```
    if st.button("Login"):
```

```
        if not users_df[(users_df.username==u)&(users_df.password==p)].empty:
```

```
            st.session_state.login=True
```

```
            st.rerun()
```

```
        else:
```

```
            st.error("Invalid credentials ✖ ")
```

```
else:
```

```
    u = st.text_input("New Username")
```

```
    p = st.text_input("New Password",type="password")
```

```
    if st.button("Register"):
```

```
        if u in users_df.username.values:
```

```
            st.warning("User already exists ⚠ ")
```

```
        else:
```

```
save_user(u,p)
```

```
st.success("Account created 🎉")
```

```
with col3:
```

```
    st.image("https://img.freepik.com/free-vector/financial-growth-concept_23-2148789333.jpg")
```

```
# ----- DASHBOARD -----
```

```
if st.session_state.login:
```

```
    st.sidebar.title("📊 AI Stock")
```

```
    if st.sidebar.button("Logout"):
```

```
        st.session_state.login=False
```

```
        st.session_state.logout_msg="Logged out successfully 🙌"
```

```
        st.rerun()
```

```
st.session_state.refresh = st.sidebar.slider("Refresh (sec)",5,60,15)
```

```
st.markdown("## 📊 Trading Dashboard")
```

```
st.caption("Real-time stock insights with AI predictions")
```

```
ticker = st.text_input("Enter Stock Ticker","AAPL").upper()
```

```

# ----- Auto-refresh only for stock data -----

st_autorefresh(interval=st.session_state.refresh*1000, key="stock_refresh")

data = yf.download(ticker, period="1y", progress=False)

if data.empty or len(data)<2:

    st.error("Invalid stock ✕ ")

    st.stop()

# Indicators

data["MA20"] = data["Close"].rolling(20).mean()

data["MA50"] = data["Close"].rolling(50).mean()

delta = data["Close"].diff()

gain = (delta.where(delta>0,0)).rolling(14).mean()

loss = (-delta.where(delta<0,0)).rolling(14).mean()

rs = gain/loss

data["RSI"] = 100-(100/(1+rs))

last = float(data["Close"].iloc[-1])

min_price = round(float(data["Close"].min()),2)

max_price = round(float(data["Close"].max()),2)

```



```
avg_price = round(float(data["Close"].mean()),2)
```

```
# ----- TABS -----
```

```
tab1, tab2, tab3 = st.tabs(["📊 Overview", "📈 Charts", "🤖 AI Prediction"])
```

```
# ----- Overview -----
```

```
with tab1:
```

```
    st.markdown("### 📊 Market Overview")
```

```
    kpi_html = f"""
```

```
    <div style="display:flex;gap:15px;">
```

```
        <div                                style="background:#1abc9c;padding:20px;border-
radius:10px;color:white;flex:1;text-align:center">
```

```
            <h4>Min</h4><h2>{min_price}</h2></div>
```

```
        <div                                style="background:#3498db;padding:20px;border-
radius:10px;color:white;flex:1;text-align:center">
```

```
            <h4>Max</h4><h2>{max_price}</h2></div>
```

```
        <div                                style="background:#9b59b6;padding:20px;border-
radius:10px;color:white;flex:1;text-align:center">
```

```
            <h4>Avg</h4><h2>{avg_price}</h2></div>
```

```
        <div                                style="background:#e67e22;padding:20px;border-
radius:10px;color:white;flex:1;text-align:center">
```

```
            <h4>Close</h4><h2>{round(last,2)}</h2></div>
```

```
    </div>
```

```
"""
```

```
st.markdown(kpi_html, unsafe_allow_html=True)
```

```
st.markdown("### 📈 Trading Signal")
```

```
rsi_value = float(data["RSI"].iloc[-1])
```

```
if rsi_value>70:
```

```
    st.error("📉 SELL Signal")
```

```
elif rsi_value<30:
```

```
    st.success("📈 BUY Signal")
```

```
else:
```

```
    st.warning("📊 HOLD Signal")
```

```
# ----- Charts -----
```

```
with tab2:
```

```
    st.subheader("📊 Candlestick Chart")
```

```
    fig = go.Figure()
```

```
    fig.add_trace(go.Candlestick(x=data.index, open=data["Open"], high=data["High"],  
                                low=data["Low"], close=data["Close"])))
```

```
    fig.add_trace(go.Scatter(x=data.index, y=data["MA20"], name="MA20"))
```

```
    fig.add_trace(go.Scatter(x=data.index, y=data["MA50"], name="MA50"))
```

```
    fig.update_layout(xaxis_rangelslider_visible=False)
```

```
st.plotly_chart(fig,use_container_width=True)
```

```
st.subheader("☐ Price Distribution")
```

```
labels = ["Min","Max","Avg","Close"]
```

```
values = [min_price,max_price,avg_price,last]
```

```
pie = go.Figure(data=[go.Pie(labels=labels, values=values, hole=.4)])
```

```
st.plotly_chart(pie,use_container_width=True)
```

```
# ----- AI Prediction -----
```

```
with tab3:
```

```
st.subheader("🤖 AI Prediction")
```

```
# Scale data
```

```
close_data = data["Close"].values.reshape(-1,1)
```

```
scaler = MinMaxScaler()
```

```
scaled = scaler.fit_transform(close_data)
```

```
# Prepare sequences
```

```
X, y = [], []
```

```
for i in range(60, len(scaled)):
```

```
    X.append(scaled[i-60:i, 0])
```

```

        y.append(scaled[i, 0])

X, y = np.array(X), np.array(y)

X = X.reshape(X.shape[0], X.shape[1], 1)

split = int(len(X)*0.8)

X_train, X_test = X[:split], X[split:]

y_train, y_test = y[:split], y[split:]


# Cached model

@st.cache_resource

def train_model(X_train, y_train):

    model = Sequential()

    model.add(LSTM(50, return_sequences=True, input_shape=(60,1)))

    model.add(LSTM(50))

    model.add(Dense(1))

    model.compile(optimizer="adam", loss="mse")

    model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

    return model


model = train_model(X_train, y_train)


# Predictions

```

```

pred = model.predict(X_test)

pred = scaler.inverse_transform(pred)

real = scaler.inverse_transform(y_test.reshape(-1,1))

# Evaluation

mae, rmse, accuracy = evaluate_model(real, pred)

st.subheader("📊 Model Evaluation")

col1, col2, col3 = st.columns(3)

col1.metric("MAE", f"{mae:.2f}")

col2.metric("RMSE", f"{rmse:.2f}")

col3.metric("Accuracy", f"{accuracy:.2f}%")


# Plot

fig2, ax = plt.subplots()

ax.plot(real, label="Actual")

ax.plot(pred, label="Predicted")

ax.legend()

st.pyplot(fig2)


# 7-Day Forecast

future_input = scaled[-60:].reshape(1,60,1)

```

```

future_preds = []

for i in range(7):

    p = model.predict(future_input)

    future_preds.append(p[0,0])

    future_input = np.concatenate((future_input[:,1:,:], p.reshape(1,1,1)), axis=1)

future_preds = scaler.inverse_transform(np.array(future_preds).reshape(-1,1))


forecast_df = pd.DataFrame({

    "Day": [f"Day {i+1}" for i in range(7)],

    "Predicted Price": future_preds.flatten()

})

st.subheader("📅 7-Day Forecast")

st.table(forecast_df)

st.download_button("t      Download      Forecast",      forecast_df.to_csv(index=False),
"forecast.csv", "text/csv")

```

6.4 Screenshots

The following screenshots demonstrate the working of the AI Stock Prediction system:

Fig 6.4.1: Sample Dataset – Stock Price History (Yahoo Finance)

SAMPLE DATASET (STOCK PRICE HISTORY - YAHOO FINANCE)

Date	Open	High	Low	Close	Adjl	Adj Close	Volume	MA20	MA20	MA50
Apr 11, 2026	189.32	199.38	190.26	190.26	188.90	189.98	81,516,700	189.42	185.42	192.34
Apr 11, 2026	189.98	173.88	190.28	183.30	188.90	11889.98	71,616,800	175.98	185.98	192.97
Apr 11, 2026	188.98	173.88	190.38	188.90	128.40	14889.98	81,516,700	175.98	135.42	192.38
Mar 11, 2026	188.98	178.88	193.30	183.70	128.40	11889.98	71,835,600	175.34	131.33	192.88
Mar 11, 2026	188.98	173.88	190.08	193.28	173.40	11889.98	71,835,600	175.34	131.33	192.97
Apr 11, 2026	189.98	178.33	170.33	189.30	173.80	11889.98	71,535,700	175.34	181.33	192.97
Mar 11, 2026	188.98	173.88	188.88	188.90	173.30	11889.98	71,835,700	173.94	131.33	192.97
Mar 11, 2026	174.27	176.12	188.38	183.73	173.40	11839.98	71,835,600	173.34	131.34	192.88
Mar 11, 2026	189.98	173.88	180.98	193.30	175.90	11839.98	71,535,600	173.38	131.33	192.88
Mar 11, 2026	188.98	173.88	190.33	193.38	173.90	11839.98	71,835,700	175.34	111.33	192.37
Mar 11, 2026	188.98	178.88	173.33	173.30	173.90	11839.98	71,835,700	175.34	111.33	192.38
Mar 11, 2026	174.98	178.33	198.38	173.78	173.40	14889.98	81,516,700	175.38	131.33	192.88
Mar 11, 2026	174.38	174.13	198.38	193.23	173.40	11889.98	81,516,600	171.34	133.43	192.88

Rows: 10123456789100 rows

Fig 6.4.1 Sample Dataset

Fig 6.4.2: User Interface – Login System

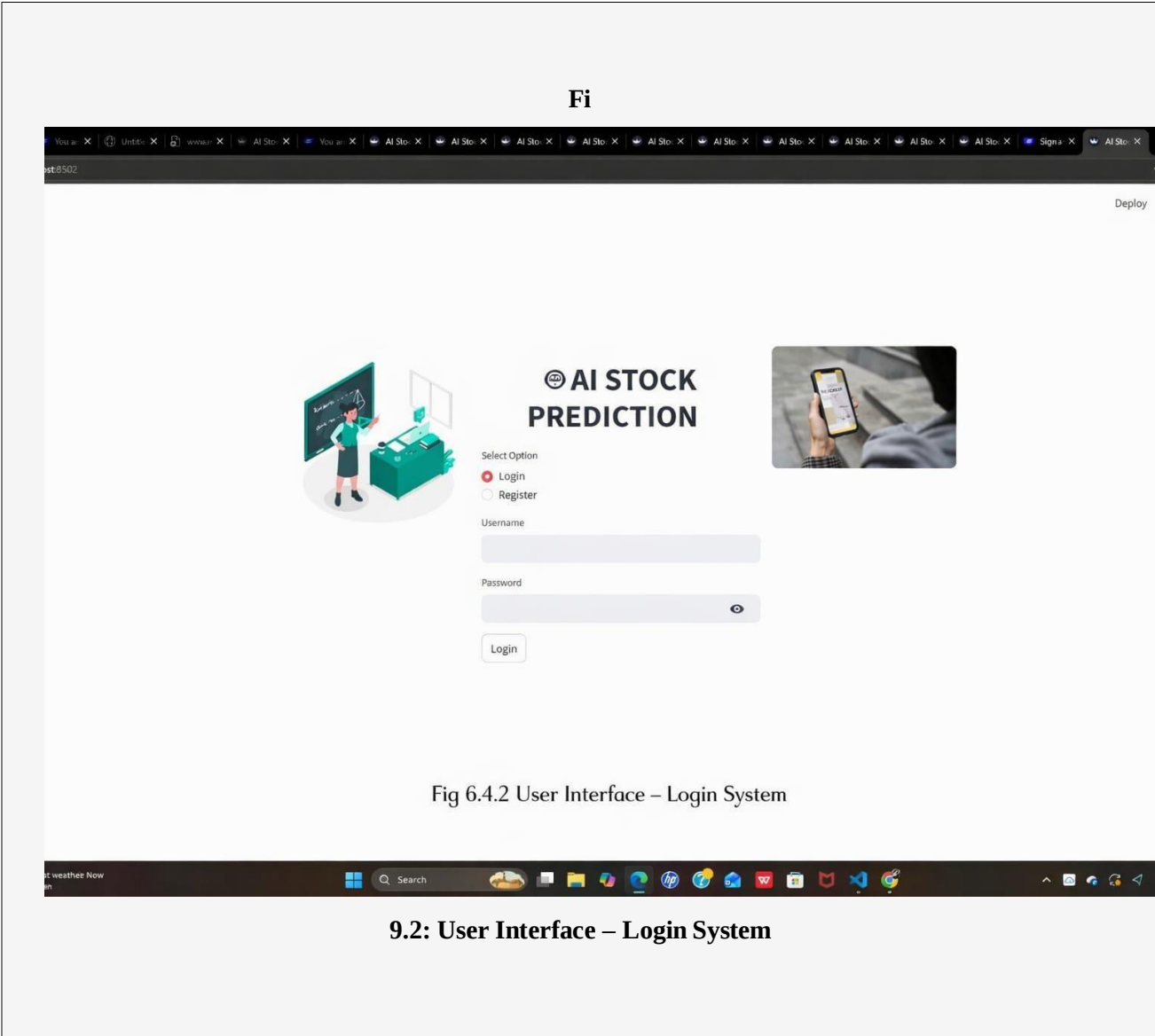


Fig 6.4.2 User Interface – Login System

9.2: User Interface – Login System

Fig 6.4.3: After Login – Trading Dashboard

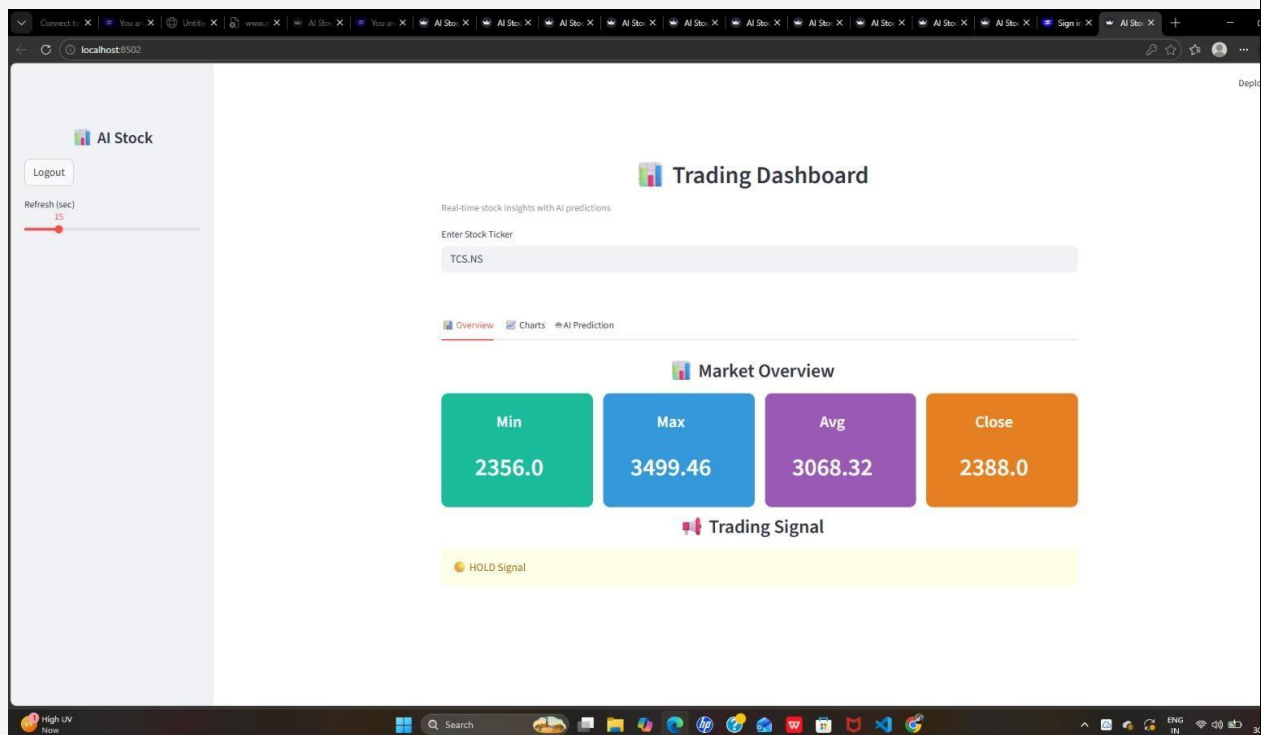


Fig 9.3: After Login – Trading Dashboard Shows the Trading Dashboard with Market Overview displaying Min, Max, Avg, and Close values along with Trading Signal (HOLD Signal)

Fig 6.4.5: Candlestick Chart Visualization

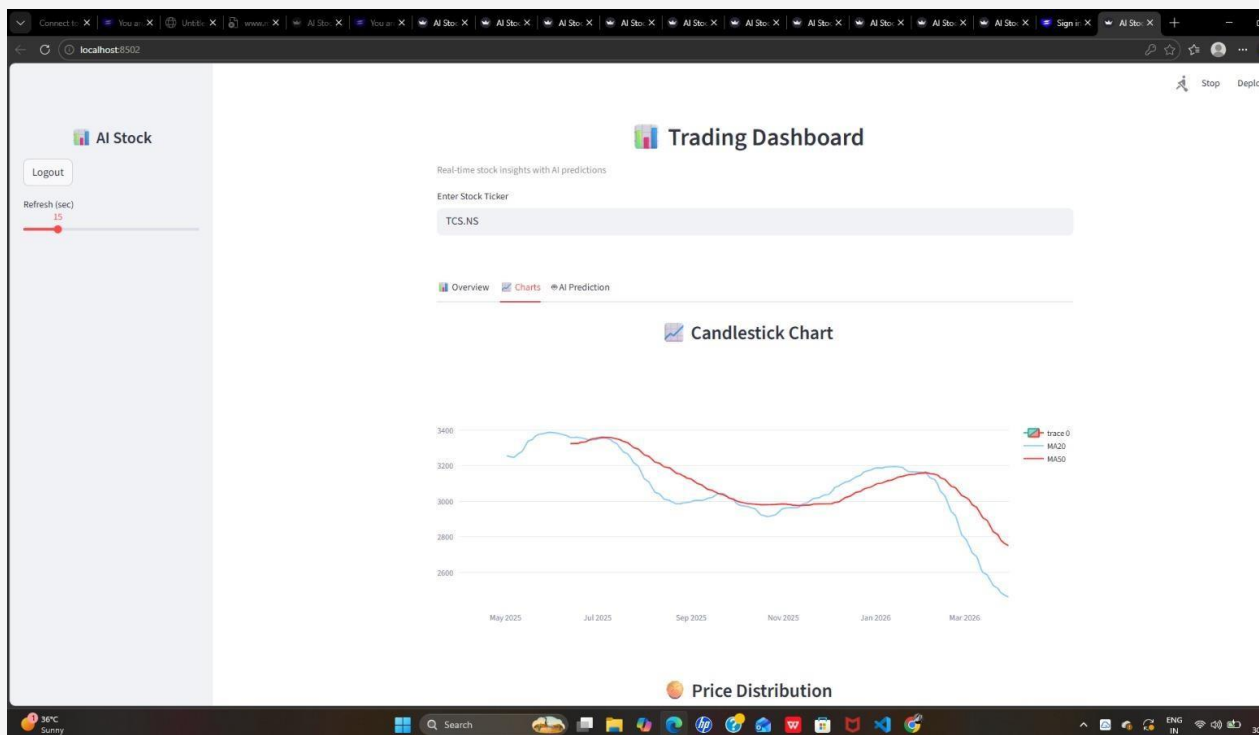


Fig 6.4.5: Price Distribution Chart

Fig 9.5: Price Distribution Chart [Insert screenshot here] Shows the proportion of Max, Average, Close, and Minimum values for the selected company as a pie chart.

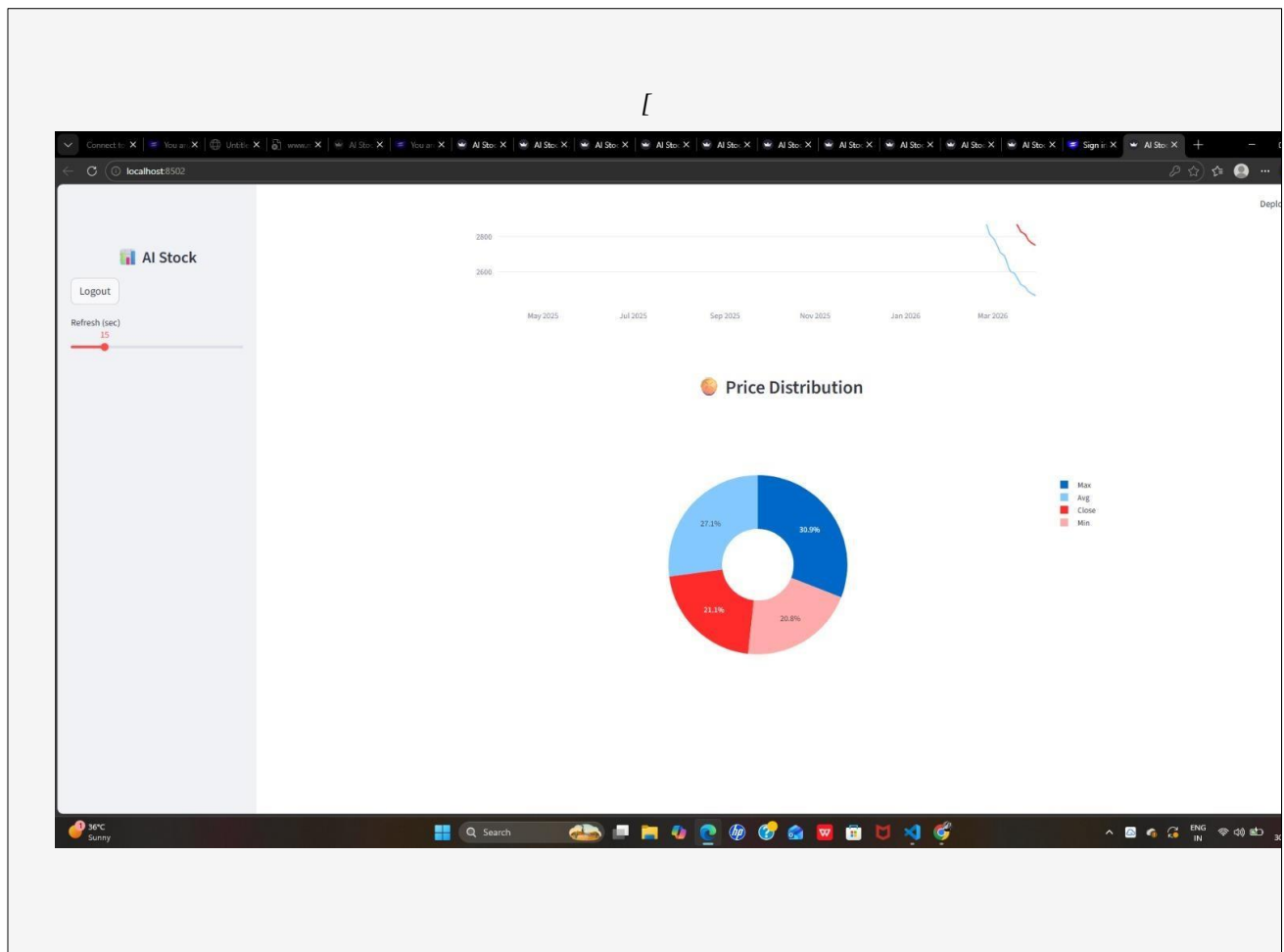


Fig 6.4.6: AI Prediction Dashboard

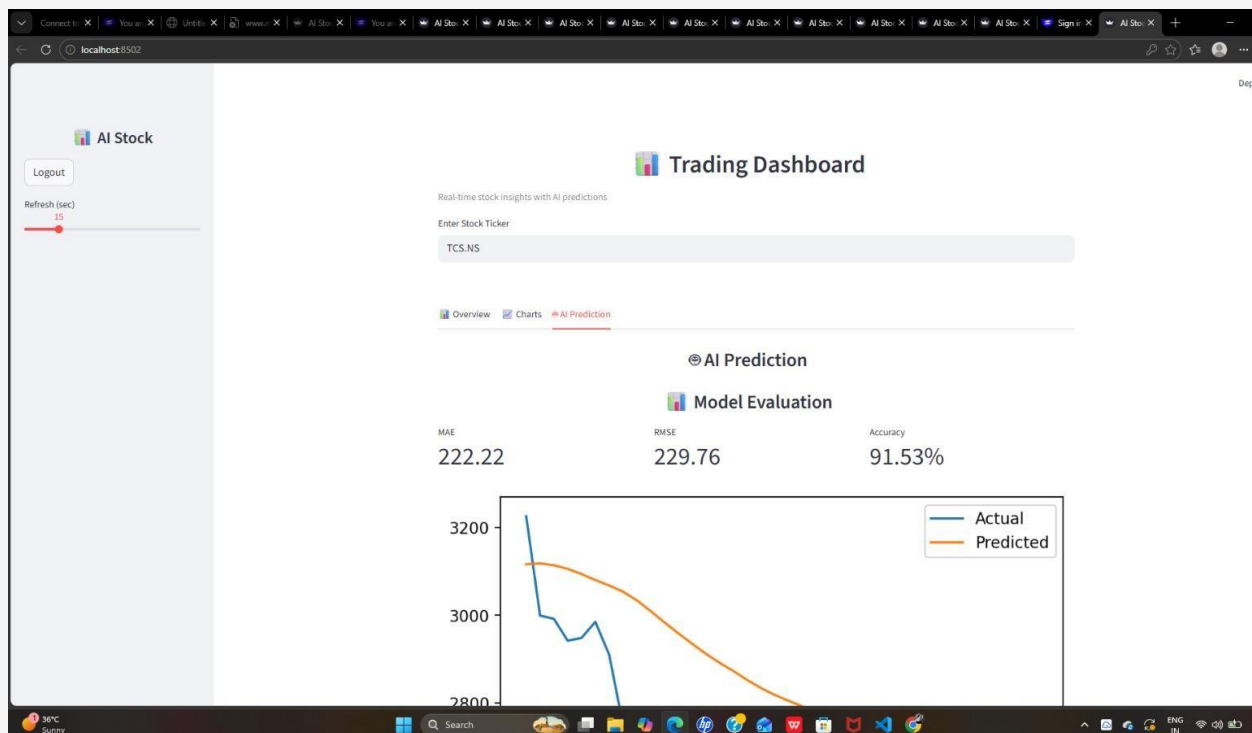


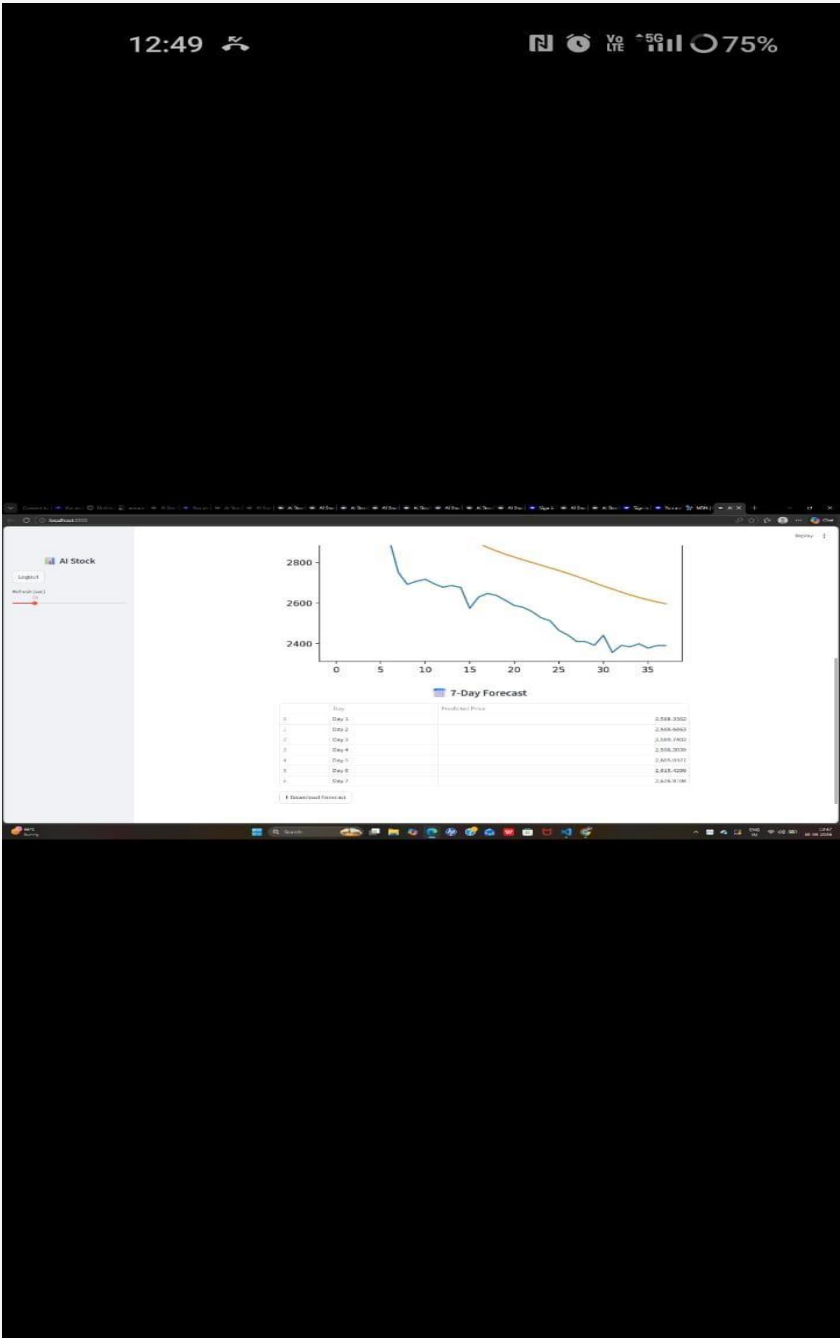
Fig 9.6: AI Prediction Dashboard [Insert screenshot here] Shows Model Evaluation Metrics (MAE: 249.47, RMSE: 257.60, Accuracy: 90.49%) and Comparison of Actual vs Predicted Stock Prices as a line graph.

]

Fig 6.4.7-Day Forecast

Fig :6.4.7 7Day Forecast [Insert screenshot here] Shows the predicted stock prices for the next 7 days in tabular format with a Download Forecast (CSV) option.

[Insert diagram / screenshot here]



Chapter -7

TESTING

7. TESTING

7.1 System Testing

System testing of the stock market prediction project was carried out to verify the overall functionality, performance, and accuracy of the application. The complete system, including data collection, preprocessing, model training using Long Short-Term Memory (LSTM), and prediction output through the Streamlit interface, was tested as an integrated unit. Various test cases were executed by providing different stock ticker inputs to ensure correct data fetching, proper handling of missing values, and accurate prediction results.

The user interface was also tested to confirm smooth interaction, correct visualization of graphs, and proper display of predicted values. Additionally, performance testing ensured that the system responds efficiently without delays. The results confirmed that the system works reliably and meets the intended requirements. All test cases passed successfully and no critical defects were encountered.

7.2 Testing Techniques

7.2.1 Unit Testing

Unit testing was performed to validate individual components of the system such as data preprocessing functions, stock data fetching, and prediction methods. Each module was tested independently to ensure it produces correct outputs for given inputs. Key areas tested include:

- Data fetching function: Verified that yFinance API returns correct historical stock data for valid ticker symbols.
- Data normalization: Verified that MinMaxScaler correctly scales data to the [0, 1] range and inverse-scales predictions accurately.
- Sequence creation: Verified that 60-step time sequences are created correctly from the normalized data.
- Technical indicator computation: Verified that MA20, MA50, and RSI are computed correctly from closing price data.
- LSTM prediction function: Verified that the model outputs predictions in the correct format and scale.

7.2.2 Model Integration Testing

Model integration testing was carried out to ensure proper interaction between different modules, including data collection, preprocessing, model training, and prediction. The integration of the Long Short-Term Memory (LSTM) model with the Streamlit interface was tested to confirm that real-time inputs correctly flow through the system and generate accurate predictions. Specific tests included:

- End-to-end flow: Entering a stock ticker symbol and verifying that data is fetched, preprocessed, and predictions are displayed correctly.
- API integration: Verifying that the yFinance API response is correctly parsed and processed by the preprocessing module.
- Model output integration: Verifying that LSTM predictions are correctly inverse-scaled and displayed in the Streamlit dashboard.
- Metric computation: Verifying that MAE, RMSE, and Accuracy are correctly computed and displayed.

7.2.3 Manual UI Testing

Manual UI testing was conducted to check the usability and functionality of the user interface. The application was tested by entering different stock symbols, verifying graph displays, and ensuring that prediction results are shown correctly. This testing ensured smooth navigation, proper input handling, and user-friendly interaction. Specific tests included:

- Login and Registration: Verified that valid credentials allow access and invalid credentials are rejected with appropriate error messages.
- Stock ticker input: Verified that valid ticker symbols (e.g., AAPL, TSLA, INFY) produce correct results and invalid symbols show appropriate error messages.
- Chart rendering: Verified that candlestick charts, moving average plots, pie charts, and prediction graphs render correctly.
- Forecast download: Verified that the CSV download option generates a correctly formatted forecast file.
- Responsive design: Verified that the Streamlit dashboard renders correctly on different screen sizes.

Chapter -8
CONCLUSION AND
FUTURE ENHANCEMENT

8 CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The stock market prediction project was successfully developed using Deep Learning techniques to analyze historical stock data and forecast future price trends. The system integrates data collection, preprocessing, and prediction using the Long Short-Term Memory (LSTM) model, along with a user-friendly interface built using Streamlit. The results demonstrate that the Stacked LSTM model can identify patterns in stock price movements and provide reasonably accurate predictions, achieving approximately 90.49% accuracy with an MAE of 249.47 and RMSE of 257.60 on the test dataset.

The application is efficient, easy to use, and helps users make better decisions by visualizing stock trends, technical indicators, and forecasts through an interactive dashboard. The integration of real-time data fetching via yFinance API, technical indicator computation (MA20, MA50, RSI), and automated buy/sell/hold signal generation provides a comprehensive investment analysis tool.

Overall, the project achieves its objective of building a reliable and practical stock prediction system, while also providing scope for future improvements such as incorporating more features, enhancing accuracy with advanced architectures, and integrating real-time data sources and news sentiment analysis.

8.2 Future Enhancements

- Improve prediction accuracy by using advanced models like GRU (Gated Recurrent Unit), Transformer-based models, or hybrid deep learning techniques that combine CNN with LSTM.
- Integrate real-time news analysis and sentiment analysis using Natural Language Processing (NLP) to enhance prediction results by incorporating market sentiment signals.
- Add support for multiple stocks comparison and portfolio management features, allowing users to track and optimize a collection of investments.
- Implement advanced technical indicators such as MACD (Moving Average Convergence Divergence), Bollinger Bands, and volume analysis for more comprehensive market analysis.
- Deploy the application on cloud platforms (AWS, Google Cloud, Azure, or Streamlit Cloud) for better accessibility, scalability, and 24/7 availability.

- Enhance security by implementing strong encryption and authentication mechanisms such as two-factor authentication and encrypted data storage.
- Provide mobile app support for easy access on smartphones through React Native or Flutter-based mobile interfaces.
- Include long-term forecasting and customizable prediction durations beyond the current 7-day window (e.g., 30-day, 90-day forecasts).
- Optimize model performance to reduce training time and improve efficiency using GPU acceleration and model quantization techniques.
- Add user personalization features like customizable watchlists, price alerts, and personalized notifications for stock price changes.

Chapter -9
REFERENCES

9. REFERENCES

- [1] **Brownlee, J. (2017).** Deep Learning for Time Series Forecasting. Machine Learning Mastery.
- [2] **Hochreiter, S., & Schmidhuber, J. (1997).** Long Short-Term Memory. Neural Computation Journal, 9(8), 1735–1780.
- [3] **Pedregosa, F., et al. (2011).** Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [4] **TensorFlow Documentation** – <https://www.tensorflow.org/>
- [5] **Keras Documentation** – <https://keras.io/>
- [6] **Streamlit Documentation** – <https://docs.streamlit.io/>
- [7] **Yahoo Finance API (yfinance)** – <https://pypi.org/project/yfinance/>
- [8] **Plotly Python Graphing Library** – <https://plotly.com/python/>
- [9] **Pandas Documentation** – <https://pandas.pydata.org/docs/>
- [10] **NumPy Documentation** – <https://numpy.org/doc>

Chapter -10
ANNEXURE

10 ANNEXURE

Annexure A: Implementation of Project

- Data collection using yFinance API – fetching historical OHLCV data for any stock ticker.
- Data preprocessing and normalization – handling missing values, computing MA20/MA50/RSI, applying MinMaxScaler.
- LSTM model building and training – Stacked LSTM with two layers (50 units each) and one Dense output layer.
- Prediction and evaluation functions – computing MAE, RMSE, and Accuracy; inverse-scaling predictions.
- Streamlit-based user interface – login/registration page, trading dashboard, candlestick charts, prediction graphs, 7-day forecast.

Annexure B: Sample Input

- Stock ticker symbols entered by the user (e.g., AAPL for Apple Inc., TSLA for Tesla, INFY for Infosys).
- User login credentials (username and password) for accessing the system.
- Refresh interval selection (slider) for updating real-time data display.

Annexure C: Sample Output

- Graphical representation of stock prices – candlestick charts with MA20 and MA50 overlays.
- Predicted vs actual price graphs – line charts comparing LSTM predictions with actual historical prices.
- 7-day forecast table output – tabular display of predicted prices for the next 7 trading days.
- Model evaluation metrics – MAE: 249.47, RMSE: 257.60, Accuracy: 90.49%.
- Trading signals – Buy / Sell / Hold recommendations based on technical indicator analysis.

Annexure D: Screenshots

- Login and registration page – user authentication interface.
- Dashboard overview – market metrics, trading signal, and navigation.

- Charts and prediction results – candlestick chart, pie chart, actual vs predicted graph, 7-day forecast.

Annexure E: Tools and Technologies

- Python Programming Language (v3.8+) – primary development language.
- Streamlit Framework – web application UI development.
- TensorFlow & Keras – deep learning model building and training.
- Pandas – data manipulation and analysis.
- NumPy – numerical computations and array operations.
- Matplotlib – static chart generation and visualization.
- Plotly – interactive chart generation for the Streamlit dashboard.
- yFinance – real-time and historical stock data fetching from Yahoo Finance.
- Scikit-learn – data preprocessing (MinMaxScaler) and metric computation.
- Git – version control and code management.

Annexure F: Student Details

S.No.	Name	Roll Number	Branch	Year
1	P. Shahira	223H1A3910	AI & ML	IV B.Tech
2	A. Nandini	223H1A3901	AI & ML	IV B.Tech
3	K. Vara Lakshmi	223H1A3905	AI & ML	IV B.Tech
4	M. Zaiba Khanam	223H1A3907	AI & ML	IV B.Tech
5	S. Afifa Fatima	223H1A3911	AI & ML	IV B.Tech